

PROGRAMA DE FORMAÇÃO ACELERADA - *Bolsa Futuro Digital*

Curso de Desenvolvimento Front-end

Exercício – GERENCIADOR DE TAREFAS

1. Contextualização

Manter uma rotina organizada pode ser um desafio quando existem diversas atividades para realizar ao longo do dia. Uma forma simples de melhorar essa organização é utilizar uma lista de tarefas, na qual seja possível registrar as atividades, definir suas prioridades e acompanhar o que já foi concluído.

Neste exercício, você deverá desenvolver uma aplicação web chamada **Gerenciador de Tarefas**, utilizando exclusivamente **HTML, CSS e JavaScript**.

A aplicação deverá permitir que o usuário cadastre suas tarefas, acompanhe seu andamento e mantenha os dados armazenados no próprio navegador. Dessa forma, as tarefas não deverão ser perdidas quando a página for atualizada ou quando o navegador for fechado e aberto novamente.

2. Objetivo

Desenvolver uma aplicação web interativa que permita ao usuário **cadastrar, visualizar, concluir, filtrar e excluir tarefas**, utilizando JavaScript para manipular os elementos da página e o localStorage para armazenar os dados.

Durante o desenvolvimento, deverão ser aplicados conceitos de:

- formulários HTML;
- manipulação do DOM;
- eventos;
- variáveis e constantes;
- estruturas condicionais;
- estruturas de repetição;
- funções;
- arrays;
- armazenamento de dados no localStorage;
- manipulação de classes CSS por meio de JavaScript.

Você será o responsável pela identidade visual de sua aplicação, ou seja, as interfaces e as respectivas formatações CSS ficarão a cargo de cada estudante.

3. Funcionalidades da aplicação

A aplicação deverá possuir, no mínimo, as seguintes funcionalidades.

3.1 Cadastro de tarefas

A página deverá apresentar um formulário para que o usuário possa cadastrar uma nova tarefa.

Cada tarefa deverá possuir minimamente:

- **Descrição:** texto que identifique a atividade;
- **Categoria:** categoria à qual a tarefa pertence;
- **Prioridade:** baixa, média ou alta;
- **Data de conclusão:** data limite para realização da tarefa.

O formulário deverá possuir um botão para cadastrar a tarefa.

Exemplo:

Descrição:	Fazer exercícios de JavaScript
Categoria:	Estudos
Prioridade:	Alta
Data:	20/09/2026
[Adicionar tarefa]	

Após o envio do formulário, a nova tarefa deverá aparecer imediatamente na lista, sem que seja necessário recarregar a página.

3.2 Listagem das tarefas

As tarefas cadastradas deverão ser apresentadas em uma área específica da página.

Cada tarefa deverá apresentar, pelo menos:

- descrição;
- categoria;
- prioridade;
- data;
- situação da tarefa.

Uma possibilidade de representação seria:

Fazer exercícios de JavaScript	
Estudos	
Prioridade: Alta	Prazo: 20/09/2026
[Concluir]	[Excluir]

A aparência da tarefa deverá ser modificada quando ela for concluída.

Por exemplo, uma tarefa concluída poderá apresentar:

- texto riscado;
- menor destaque visual;
- alteração de cor;
- indicação textual de "Concluída".

A forma como isso será implementado fica a seu critério.

3.3 Concluir uma tarefa

Cada tarefa deverá possuir uma forma de indicar que ela foi concluída.

Ao realizar essa ação, a aplicação deverá:

1. alterar o estado da tarefa;
2. atualizar sua representação visual;
3. atualizar os indicadores da aplicação;
4. salvar a alteração no localStorage.

Também deverá ser possível desfazer a conclusão de uma tarefa, retornando-a para o estado de pendente.

3.4 Excluir uma tarefa

Cada tarefa deverá possuir uma opção para exclusão.

Antes de excluir definitivamente uma tarefa, a aplicação deverá solicitar uma confirmação ao usuário.

Após a confirmação:

- a tarefa deverá desaparecer da lista;
- os indicadores deverão ser atualizados;
- a alteração deverá ser salva no localStorage.

4. Filtros

A aplicação deverá permitir que o usuário filtre as tarefas apresentadas.

Deverão existir, no mínimo, os seguintes filtros:

- **Todas**
- **Pendentes**
- **Concluídas**

Também deverá existir um filtro por categoria.

Exemplo:

Situação: [Todas ▼]

Categoria: [Todas ▼]

Ao alterar um filtro, a lista deverá ser atualizada dinamicamente, sem recarregar a página.

Os filtros deverão afetar apenas a visualização. Uma tarefa que não esteja sendo exibida devido a um filtro não deverá ser excluída dos dados armazenados.

5. Indicadores

A aplicação deverá apresentar um pequeno resumo da situação das tarefas.

No mínimo, deverão ser exibidas as seguintes informações:

Total de tarefas: 8

Tarefas pendentes: 5

Tarefas concluídas: 3

Esses valores deverão ser atualizados automaticamente sempre que uma tarefa for:

- adicionada;
- concluída;
- excluída.

6. Armazenamento no navegador

As tarefas deverão ser armazenadas utilizando o **Local Storage do navegador**.

Isso significa que, depois de cadastrar algumas tarefas, o usuário poderá:

1. atualizar a página;
2. fechar o navegador;
3. abrir novamente a aplicação;

e as tarefas anteriormente cadastradas deverão continuar disponíveis.

Ao iniciar a aplicação, o JavaScript deverá verificar se existem tarefas armazenadas e, caso existam, carregá-las para a interface.

As alterações realizadas posteriormente também deverão ser refletidas no armazenamento.

Importante: o localStorage armazena dados como texto. Portanto, você deverá definir uma estratégia para transformar os dados das tarefas em texto para armazenamento e recuperar esses dados posteriormente.

7. Validação do formulário

A aplicação deverá impedir o cadastro de tarefas inválidas.

No mínimo:

- a descrição não poderá estar vazia;
- a categoria deverá ser selecionada;
- a prioridade deverá ser selecionada;
- a data deverá ser informada.

Quando houver algum problema no preenchimento, o usuário deverá receber uma mensagem indicando o que precisa ser corrigido.

A validação deverá ocorrer utilizando JavaScript e/ou os recursos de validação disponíveis nos elementos de formulário HTML.

8. Regras adicionais

A aplicação deverá obedecer às seguintes regras:

1. Uma tarefa recém-cadastrada deverá iniciar como **pendente**.
2. Uma tarefa concluída não deverá deixar de existir; apenas seu estado deverá ser alterado.
3. A exclusão deverá remover efetivamente a tarefa dos dados armazenados.
4. Os filtros não poderão modificar ou excluir os dados originais.
5. As informações apresentadas nos indicadores deverão corresponder aos dados atualmente armazenados.
6. Todas as alterações realizadas pelo usuário deverão ser refletidas na interface imediatamente.
7. Os dados deverão permanecer disponíveis após a atualização da página, por meio do local storage.

9. Requisitos de JavaScript

O código deverá utilizar funções para organizar as diferentes responsabilidades da aplicação.

Por exemplo, espera-se que existam funções responsáveis por tarefas como:

- adicionar uma tarefa;
- exibir as tarefas;
- atualizar os indicadores;
- alterar o estado de uma tarefa;
- excluir uma tarefa;
- filtrar tarefas;
- salvar os dados;
- carregar os dados.

Os nomes e a organização das funções ficam a seu critério.

O objetivo não é determinar uma implementação única, mas demonstrar que você consegue **dividir um problema maior em pequenas partes e utilizar funções para resolvê-las**.

10. Interação com o DOM

As tarefas deverão ser inseridas na página dinamicamente utilizando JavaScript.

Não será suficiente deixar todas as tarefas previamente escritas no HTML.

Por exemplo, quando o usuário cadastrar:

Estudar HTML

o JavaScript deverá criar ou atualizar os elementos necessários para que essa tarefa apareça na lista.

Da mesma forma, quando uma tarefa for excluída, sua representação deverá ser removida ou atualizada dinamicamente.

11. Eventos

A aplicação deverá utilizar eventos JavaScript para responder às ações do usuário.

Deverão ser utilizados, conforme a necessidade do projeto, eventos como:

- envio do formulário;
- clique em botões;
- alteração de filtros;
- alteração de campos do formulário.

O comportamento da aplicação deverá ocorrer como resposta às ações do usuário, sem a necessidade de recarregar a página a cada operação.

12. Organização visual

A aplicação deverá possuir uma interface organizada e consistente.

A página deverá apresentar, no mínimo:

1. um título;
2. uma área para cadastro;
3. uma área para filtros;
4. uma área de resumo/indicadores;
5. uma área para apresentação das tarefas.

A definição das cores, tipografia, espaçamentos, ícones e demais elementos visuais ficará a seu critério

A interface deverá ser suficientemente clara para que um usuário consiga compreender:

- como cadastrar uma tarefa;
- quais tarefas estão pendentes;
- quais foram concluídas;
- como filtrar a lista;
- como excluir uma tarefa.

13. Situação inicial

Ao abrir a aplicação pela primeira vez, não haverá tarefas cadastradas.

Nesse caso, a área de tarefas deverá apresentar uma mensagem apropriada, como:

Nenhuma tarefa cadastrada.

Adicione uma tarefa para começar!

Essa mensagem deverá deixar de ser exibida quando existir pelo menos uma tarefa.

Caso existam tarefas armazenadas no localStorage, elas deverão ser carregadas automaticamente.

14. Desafios opcionais

Para estudantes que desejarem ampliar a aplicação, poderão ser implementadas funcionalidades adicionais. Algumas possibilidades são:

- pesquisa por texto;
- ordenação por prioridade;
- ordenação por data;
- contador de tarefas atrasadas;
- botão "Excluir todas as concluídas";
- possibilidade de editar uma tarefa;
- filtro por prioridade;
- indicador de percentual de tarefas concluídas;
- utilização de diferentes estilos para cada nível de prioridade;
- possibilidade de definir uma descrição mais detalhada para a tarefa.

17. Entrega

É desejável que você crie o projeto no git, preferencialmente gerenciando o versionamento do seu projeto com esta ferramenta. O projeto deverá ser entregue contendo, no mínimo:

gerenciador-tarefas/

```
|
|— index.html
|— css/
|   |— style.css
|— js/
|   |— script.js
```

O projeto deverá funcionar abrindo o arquivo index.html no navegador, podendo ter tantas páginas html quanto você julgue necessário.

O código deverá ser organizado e possuir nomes de variáveis, funções e elementos que permitam compreender sua finalidade. Comentários sempre são desejáveis.